

Descrição geral do trabalho:

O objetivo do trabalho é implementar uma estrutura van Emde Boas com espaço linear no número de elementos, ou seja, usando tabela de dispersão com table doubling/halving. A estrutura deve ser feita para o tamanho de palavra de 32 bits. A entrada do programa é um arquivo de texto (.txt), que deve ser recebido como argumento do programa, com várias operações sobre a estrutura (uma por linha) e a saída será o resultado das operações impressos no próprio terminal que estiver sendo utilizado.

Operações:

- **Inclusão:** Uma operação de inclusão será identificada por uma linha escrito INC seguido de um espaço e depois um inteiro. Este elemento deve ser incluído na estrutura de dados e nada precisa ser impresso.
Exemplo de linha de inclusão:
INC 13
- **Remoção:** Uma operação de remoção será identificada por uma linha escrito REM seguido de um espaço e depois um inteiro. Este valor deve ser removido da estrutura, se existir, e nada precisa ser impresso.
Exemplo de linha de remoção:
REM 42
- **Sucessor:** Uma operação de sucessor será identificada por uma linha escrito SUC seguido de um espaço e depois um inteiro. O sucessor do número x é o menor valor na estrutura que é estritamente maior que x . Esta operação gera como resultado o sucessor do inteiro dado como entrada. Caso não exista valor na estrutura maior que o inteiro fornecido, o sucessor será +INF.
Exemplo de linha de sucessor:
SUC 50
Exemplo dessa linha na impressão de saída:
SUC 50
80
- **Predecessor:** Uma operação de predecessor será identificada por uma linha escrito PRE seguido de um espaço e depois um inteiro. O predecessor do número x é o maior valor na estrutura que é estritamente menor que x . Esta operação gera como resultado o predecessor do inteiro dado como entrada. Caso não exista valor na estrutura menor que o inteiro fornecido, o predecessor será -INF.
Exemplo de linha de predecessor:
PRE 50
Exemplo dessa linha na impressão de saída:
PRE 50
21
- **Imprimir:** Uma operação de impressão será identificada por uma linha escrito IMP. A impressão de uma van Emde Boas oferece uma visão a partir do primeiro nível da estrutura. Neste caso, você deve imprimir o valor mínimo da estrutura, os valores que estão nos clusters não vazios.
Exemplo de linha de impressão:
IMP
Exemplo dessa linha na impressão de saída:

IMP

Min: 102, C[0]: 268, 322, 14756, C[3]: 456, 728, 152, C[65535]: 0, 65535

Grupos e Entrega:

- Cada equipe da graduação pode ser formada por até 3 membros.
- O trabalho na pós-graduação é individual.

O envio do trabalho deve conter:

- O seu código-fonte
- Um arquivo **makefile** que possua funcionalidades para compilar (**build**) e executar (**run**) o seu programa.
 - Lembre-se que seu programa deve receber um arquivo de texto (.txt) como entrada.
 - Logo, o comando para a funcionalidade **run** deve ser capaz de receber um argumento que é o endereço de um arquivo de texto que será a entrada do programa.
- Um arquivo **README.md** de descrição do trabalho que indica:
 - A linguagem de programação usada (incluindo a versão usada)
 - Como rodar o seu programa
 - Descrição das funções e estruturas (equivalente ao struct em C) que são usadas no seu código
 - Em quais arquivos cada uma dessas funções e estruturas se encontram.

Exemplo do **makefile** (com todos os arquivos no mesmo diretório):

```
makefile
1  INPUT = entrada.txt
2
3  build: main.cpp
4      g++ -o programa.exe main.cpp
5
6  run: programa.exe
7      ./programa.exe $(INPUT)
8
```

Nesse exemplo, temos 2 comandos para o make: *build* e *run*

O comando **make build** irá executar a linha 4 (abaixo da definição do *build*). Ele precisa que o arquivo **main.cpp** exista no diretório (por isso está sendo especificado depois dos dois pontos na linha 3). Ele irá compilar o conteúdo do arquivo no executável **programa.exe**.

Já o comando **make run** irá executar a linha 7. Ele precisa que o arquivo **programa.exe** exista no diretório (ou seja, só pode ser executado depois do *build*). Ele irá executar o programa, passando como parâmetro o endereço de um arquivo de texto (assumindo que o código feito execute recebendo um arquivo .txt como argumento de entrada).

O **INPUT** é uma variável (de string) definida dentro do arquivo do **makefile**. Por padrão definimos ele como **entrada.txt**. Se executarmos apenas o comando **make run**, ele irá executar a linha 7 substituindo o **\$(INPUT)** pela string salva nele, e com isso, rodará o **programa.exe** passando o arquivo **entrada.txt** como entrada (se ele existir).

Se quisermos usar um arquivo com nome diferente do padrão, basta executar o comando ***make run INPUT="nome do arquivo"***. Por exemplo, com um arquivo de nome ***entrada2.txt***:

```
make run INPUT=entrada2.txt
```

O que estamos fazendo é atribuir outra *string* à variável, antes de rodar o comando. Assim, o comando irá rodar o ***programa.exe*** passando como argumento um arquivo com o nome (***entrada2.txt***) que colocamos no comando.

Testes:

Sugestão para realização de testes (com todos os arquivos no mesmo diretório):

Para cada arquivo de entrada (***entrada.txt*** por exemplo), mantenha um arquivo com a saída esperada para essa entrada (***saida_esperada.txt*** por exemplo).

Então, após compilar seu programa (***programa.exe*** por exemplo), rode o seguinte comando no terminal (no diretório onde os arquivos estão):

```
./programa.exe entrada.txt > saida.txt
```

Isso irá copiar todas as linhas que foram impressas durante a execução do programa usando o arquivo ***entrada.txt*** como entrada, e irá salvá-las em um novo arquivo ***saida.txt*** no mesmo diretório.

Após isso, rode o seguinte comando no terminal (no diretório onde os arquivos estão):

```
diff saida.txt saida_esperada.txt > diferencas.txt
```

(No terminal cmd do Windows, use o comando ***fc*** em vez de ***diff***)

Isso irá comparar os 2 arquivos e gerar um arquivo ***diferencas.txt*** no mesmo diretório, que irá conter todas as diferenças entre a saída obtida e a saída esperada para a entrada usada.

Exemplo:

Seja o arquivo **entrada.txt** que possui as seguintes linhas de operações:

```
INC 655364
INC 65535
SUC 100000
IMP
```

E seja o arquivo **saida_esperada.txt**, que possui as seguintes linhas esperadas para a execução do arquivo *entrada.txt*:

```
SUC 100000
655364
IMP
Min: 65535, C[10]: 4
```

Durante a execução do programa foi passado o arquivo *entrada.txt* como entrada, e, usando o comando mostrado, ele gerou o arquivo **saida.txt** com as seguintes linhas:

```
SUC 100000
655364
IMP
Min: 65535, C[20]: 4
```

Ao executar o comando *diff* mostrado, para comparar a saída obtida com a saída esperada, isso irá gerar um arquivo **diferencas.txt** com as seguintes linhas:

```
4c4
< Min: 65535, C[20]: 4
---
> Min: 65535, C[10]: 4
```

(Se usar o comando **fc** do Windows, o arquivo será um pouco diferente, mas a mesma ideia ainda se aplica)

Essas linhas significam que houve diferença na linha 4 entre os dois arquivos.

Isso indica que provavelmente existe um erro durante a segunda operação INC, ou na própria operação de impressão IMP, realizadas durante a execução do programa.